# QUALITY PLAN

Livrable du projet ESP-901

Réalisé par Justin MBODJE, Yasser MEKHFI, Jérémy COQUELIN, Mohamed FOFANA, Mattéo COGAN, Tristan CHARDES, Romain CROCQ-MARCHAND

Établissement de formation : EPITECH RENNES

Promotion 2026

# Sommaire

**1. Définir l'organisation de l'équipe....3**
Membre de l'équipe....3
Rôle....3
Organisation de la communication....4
Organisation du travail....4
**2. Définir le processus de développement....5**
Méthodologie de travail....5
Cycle de développement....5
**3. Définir les normes et bonnes pratiques....6**
Conventions de codage....6
1.1 Nommage des fonctions et variables....6
1.2 Architecture....6
Versionnage du code....6
Documentation....7
1.1 Documentation du code....7
1.2 Documentation de l'api....7
**4. Définir la stratégie de tests....8**
Types de tests....8
Outils de test....8
Couverture....8
**5. Définir la gestion des livraisons....9**
Definition Of Done branch stack....9
Definition Of Done branch main....9
Gestion des versions....9
Déploiement continu....9
Monitoring....10
**6. Définir les besoins en matériel et logiciel....11**
Matériel....11
Logiciel....11

# 1.Définir l'organisation de l'équipe

## Membre de l'équipe

| Prénom | Rôle |
|---|---|
| Mohamed | Chef de projet<br>Dev Mobile |
| Justin | Chef de projet en second<br>Lead IA |
| Romain | Lead Devops |
| Yasser | Lead Back<br>IA |
| Tristan | Lead Dev Front |
| Jérémy | Lead Mobile |
| Mattéo | Dev Front |

## Rôle

**Chef de projet**

Le chef de projet est la personne qui possède une connaissance globale du projet. Elle gère l'équipe et s'assure du respect de la qualité de livraisons (deliveries), du respect des dates de rendu. Il représente l'équipe lors des réunions avec le client. Il doit faire le point avec les différents lead afin de mesurer l'avancée du projet.

**Chef de projet en second**

Le chef de projet en second représente l'équipe en cas de l'absence du chef de projet lors des réunions avec le client. Il s'assure du respect de certaines choses que le chef de projet lui a délégué.

## Organisation de la communication

Pour proposer un projet de qualité notre équipe se doit de communiquer de manière optimale afin de faire circuler au mieux les informations. Pour ce faire nous mettons en place un serveur Discord permettant de communiquer de manière textuelle sur des sujets spécifiques dans des salons dédiés. Des communications orales sont également possibles.

L'organisation de réunions suivant un rythme bi-mensuel permet de réunir l'équipe. Lors de ces réunions l'ordre du jour est annoncé et les différentes parties concernées prennent alors la parole. Le chef de projet rappelle les objectifs, les leads présente l'avancé de leur partie et parle des difficultées rencontrées s'il y en a. Ensuite, les développeurs peuvent faire part de leur remarque.

## Organisation du travail

Notre projet se base sur la méthodologie de travail agile "scrum". Nous avons choisi cette méthode car elle permet de produire des nouvelles versions rapidement apportant des correctifs et améliorations de manière constante au projet. Cette méthode est aussi très adaptée pour une équipe de notre taille.

La répartition des tâches, elle, s'effectue à travers un Jira. Sur cet outil les différentes parties du projet sont découpées en tâche et sous tâche. Chaque personne peut consulter les tâches qui lui sont attribuées, changer le statut de la tâche (non commencée, en cours et terminée).

# 2.Définir le processus de développement

## Méthodologie de travail

- **Méthode Agile SCRUM**
  - Sprints de 2 semaines
  - Démo en fin de sprint
  - Réunion de planification et rétrospective à chaque sprint

## Cycle de développement

1. **Planification**
   - Rédaction de spécifications fonctionnelles et techniques
   - Création de tickets de tâche
2. **Développement**
   - Création de branches de fonctionnalités (`feature/`, `bugfix/`, etc.)
   - Suivi avec GitHub + Jira
3. **Tests**
   - Tests unitaires, intégration, e2e
4. **Code Review**
   - Merge request soumise et approuvée par un dev
   - Code approuvé par Code Rabbit
5. **Validation**
   - Tests manuels + automatisés + CI/CD
6. **Déploiement**
   - Sur environnement de production

# 3.Définir les normes et bonnes pratiques

## Conventions de codage

Le développement du **back-office web** se fera en JavaScript avec le framework **React** et Vite. Le développement du **backend** (API) se fera en JavaScript/TypeScript avec **Node.js** et le framework **Fastify**. Le développement de **l'application mobile** (produit principal) se fera en **Flutter** (langage Dart).

### 1.1 Nommage des fonctions et variables

Pour le nommage des fonctions et variable l'usage du camelcase est requis.

Exemple :

masuperfonction => maSuperbeFonction

### 1.2 Architecture

api/ → Routes API Fastify.

api/ → Fichiers backend autre que les routes API

api/ → Modèles pour la base de données.

api/ → Logique métier.

api/auth → Authentification

## Versionnage du code

Pour le versionnage du code, la plateforme GitHub sera utilisée. La branche main doit contenir une version du projet fonctionnel. Pour pousser sur cette branche une merge request doit être validée par le chef de projet et le lead de la stack.

Chaque stack possède sa propre branche où le contenu doit être fonctionnel. Pour pousser sur cette branche une merge request doit être validée par le lead de la stack, le dev qui a fait la merge request et Code Rabbit.

# Documentation

## 1.1 Documentation du code

Chaque stack doit rédiger et maintenir à jour une documentation au format markdown sur sa branche principale. Elle doit référencer chaque fonction et expliquer ses paramètres.

## 1.2 Documentation de l'api

L'API devra être documentée avec Swagger.

# 4.Définir la stratégie de tests

## Types de tests

- **Tests unitaires :** chaque fonction critique
- **Tests d'intégration :** modules entiers (API + DB)
- **Tests manuels :** QA de dernière minute

## Outils de test

- **Front/Back :** Vitest
- **CI/CD :** Exécution automatisée des tests avant toute merge

## Couverture

- Objectif de **80% de couverture de code sur le backend**
- Rapports de couverture générés à chaque pipeline CI

# 5.Définir la gestion des livraisons

## Definition Of Done branch stack

- 100% des tests passés
- Merge request approuvé par le lead de la stack
- La documentation est mise à jour

## Definition Of Done branch main

- 100% des tests passés
- Merge request approuvé par le chef de projet
- La documentation est mise à jour

## Gestion des versions

Les versions doivent être taguée de la manière suivante : x.y.z où, y la version mineur et z un patch.

x (Version majeure) : Ajout majeurs (changement majeur pouvant altérer la compatibilité de l'ancienne version)

y (Version mineure) : Ajout de nouvelles fonctionnalités sans casser la compatibilité existante (nouvelle route API, changement front léger)

z (Patch) : Corrections de bugs et améliorations mineures sans changement fonctionnel (ex : fix d'un bug, correction de sécurité)

## Déploiement continu

Le déploiement de l'application sera complètement automatisée à travers une pipeline CI/CD. La pipeline sera utilisée à travers un runner Github. A travers différents jobs,

elle s'occupera de la validation des tests, la création des images docker, la mise en ligne de l'image docker sur le Digital Ocean et le déploiement sur un cluster Kubernetes.

## Monitoring

La mise en place d'un grafana permettra de consulter les logs systèmes (serveur) et applicative (produite par l'application) et de vérifier le bon fonctionnement de l'infrastructure et de l'application, avec Prometheus pour la collecte des métriques. Elles permettront de vérifier le bon fonctionnement de l'infrastructure et de l'application.

# 6.Définir les besoins en matériel et logiciel

## Matériel

1. **Ordinateurs** – Capables de supporter l'environnement de développement choisi.
2. **Serveurs** – Si une infrastructure spécifique est nécessaire pour le développement ou les tests.
3. **Équipements spécifiques** – 2 téléphones (android et iOS) et un macbook.
4. **Coût estimé** – Évaluer le budget nécessaire pour ces composants.

## Logiciel

1. **Environnement de développement (IDE, éditeur de code)** – Exemple : VS Code, WebStorm, Windsurf.
2. **Langages et frameworks** – React.js, NodeJS, Flutter, Fastify.
3. **Outils de gestion de configuration** – Git, GitHub.
4. **Outils de tests** – Vitest.
5. **Outils de gestion de projet** – Jira, Notion, Github Project.
6. **Outils CI/CD** – GitHub Actions, Terraform, Docker.